

Base de Datos Vectoriales

SEMANA 10

Heider Sanchez
hsanchez@utec.edu.pe

5.



Modelos de Recuperación de Texto

Índice Invertido Optimizado

Optimización de la Indexación

- ¿Como construimos un índice eficiente y escalable?
- ¿Qué estrategias puedo usar con memoria principal limitada?
- ¿Cómo podemos hacer uso de la memoria secundaria?



Optimización de la Indexación

- **Scaling index construction**

- Construir un índice en memoria principal no es escalable.
 - No puede guardarse toda la colección en la memoria => ordenar y luego volver a escribir.
- ¿Cómo puedo escribir un índice para colecciones muy grandes?
- Tomando en cuenta las restricciones del hardware. . .
 - Memoria, disco, velocidad, etc.
- Transferir datos por bloques del disco a la memoria es más rápido que transferir muchos fragmentos pequeños.

Blocked Sort-Based Indexing (BSBI)

- 8 bytes cada entrada (*termID*, *docID*)
- Estos se generan a medida que analizamos cada documento
- Luego se debe ordenar las entradas mediante el termID.
 - Ejemplo 100 entradas.
 - Si definimos un bloque equivalente a 10 entradas.
 - Se require 10 bloques
- Idea básica del algoritmo:
 - Acumular postings para cada bloque, ordenar, y escribir en el disco
 - Un **posting** incluye el DocID, Frecuencia (TF) y/o Posiciones.
 - Luego mezclar los bloques en una estructura grande y ordenada.

Blocked Sort-Based Indexing (BSBI)

BSBIINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do  $n \leftarrow n + 1$ 
4       $block \leftarrow \text{PARSENEXTBLOCK}()$ 
5       $\text{BSBI-INVERT}(block)$ 
6       $\text{WRITEBLOCKTODISK}(block, f_n)$ 
7   $\text{MERGEBLOCKS}(f_1, \dots, f_n; f_{\text{merged}})$ 
```

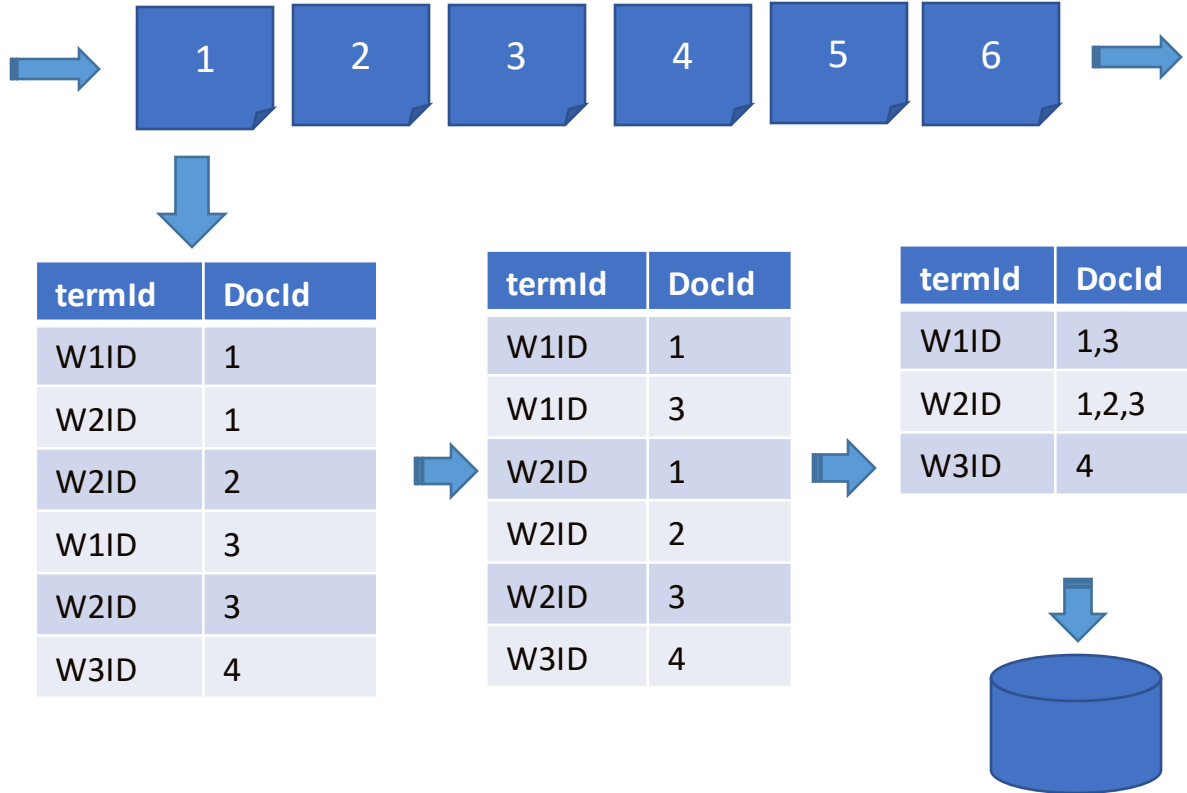
termId	DocId
W1ID	1
W2ID	1
W2ID	2
W1ID	3
W2ID	3
W3ID	4

Sort and Build a
local index

termId	DocId
W1ID	1,3
W2ID	1,2,3
W3ID	4

Blocked Sort-Based Indexing (BSBI)

- Indices Locales



Blocked Sort-Based Indexing (BSBI)

- Indices Locales

Dictionary (RAM)

Term	TermID
Term1	W1ID
Term2	W2ID
Term3	W3ID
Term4	W4ID
Term5	W5ID
Term6	W6ID
Term7	W7ID

Local Indexes
(DISK BLOCKS)

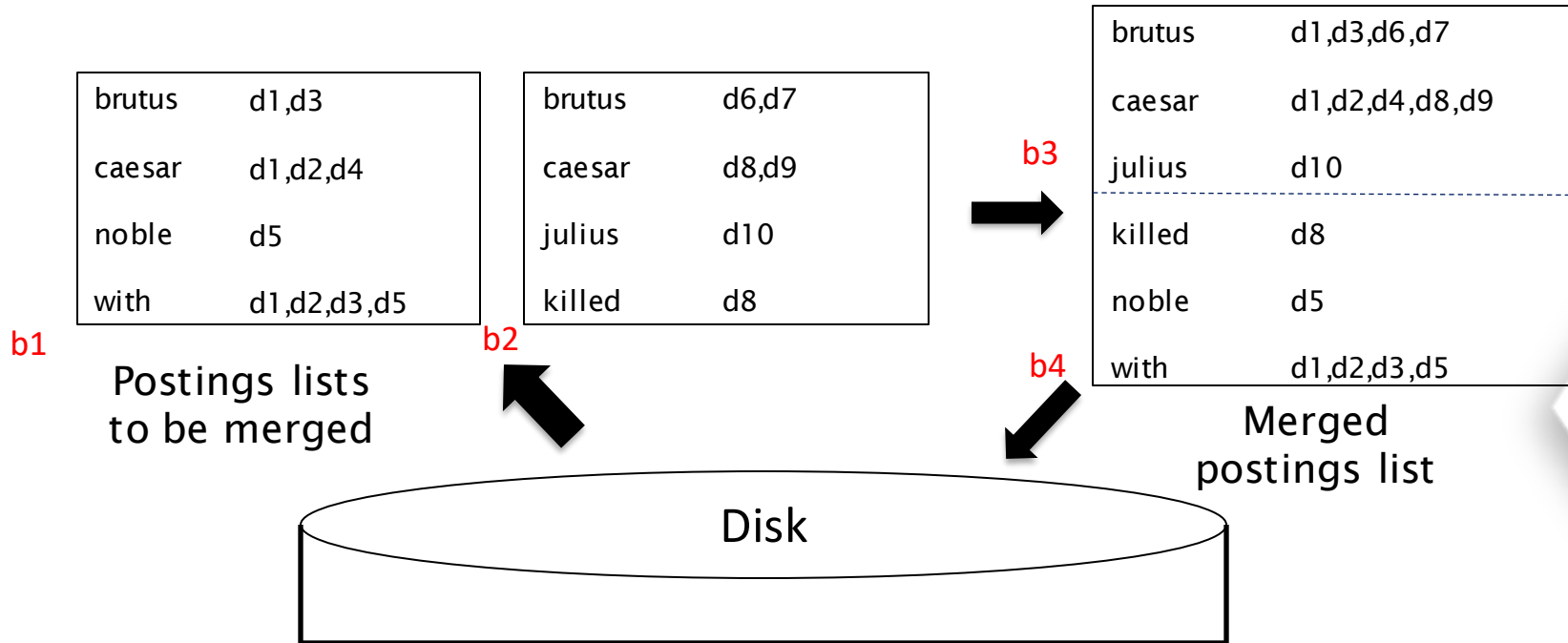
termId	DocId	termId	DocId	termId	DocId	termId	DocId	termId	DocId
W1ID	1,3	W2ID	4,5	W1ID	7, 8	W2ID	9	W2ID	11
W2ID	1,2,3	W4ID	6	W3ID	6, 7	W4ID	10,11	W3ID	11, 12
W3ID	4	W5ID	4,5,6	W6ID	6,8	W7ID	9,10	W7ID	11

Los mismos términos pueden aparecer en mas de un bloque

¿Como mezclar los bloques?

Blocked Sort-Based Indexing (BSBI)

- ¿Como mezclar los bloques?



Blocked Sort-Based Indexing (BSBI)

- ¿Como mezclar los bloques?

Dictionary (RAM)

Term	TermID
Term1	W1ID
Term2	W2ID
Term3	W3ID
Term4	W4ID
Term5	W5ID
Term6	W6ID
Term7	W7ID

Local Indexes
(DISK BLOCKS)

termId	DocId	termId	DocId	termId	DocId	termId	DocId	termId	DocId
W1ID	1,3	W2ID	4,5	W1ID	7, 8	W2ID	9	W2ID	11
W2ID	1,2,3	W4ID	6	W3ID	6, 7	W4ID	10,11	W3ID	11, 12
W3ID	4	W5ID	4,5,6	W6ID	6,8	W7ID	9,10	W7ID	11

MergeBlocks (L1)

MergeBlocks (L1)

termId	DocId	termId	DocId	termId	DocId	termId	DocId	termId	DocId
W1ID	1,3	W3ID	4	W1ID	7, 8	W4ID	10,11	W2ID	11
W2ID	1,2,3,4,5	W4ID	6	W2ID	9	W6ID	6,8	W3ID	11, 12
		W5ID	4,5,6	W3ID	6, 7	W7ID	9,10	W7ID	11

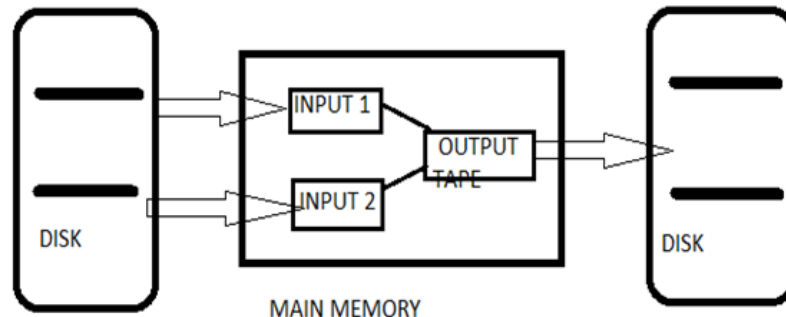
MergeBlocks (L2)



Blocked Sort-Based Indexing (BSBI)

- **¿Como mezclar los bloques?**

- Si se considera 10 bloques de 10 registros.
- Se puede hacer mezclas binarias, con un árbol de mezcla de $\log_2 10 = 4$ niveles.
- Durante cada capa, la lectura en memoria se ejecuta en bloques de 10M, se fusiona, se escribe de nuevo.

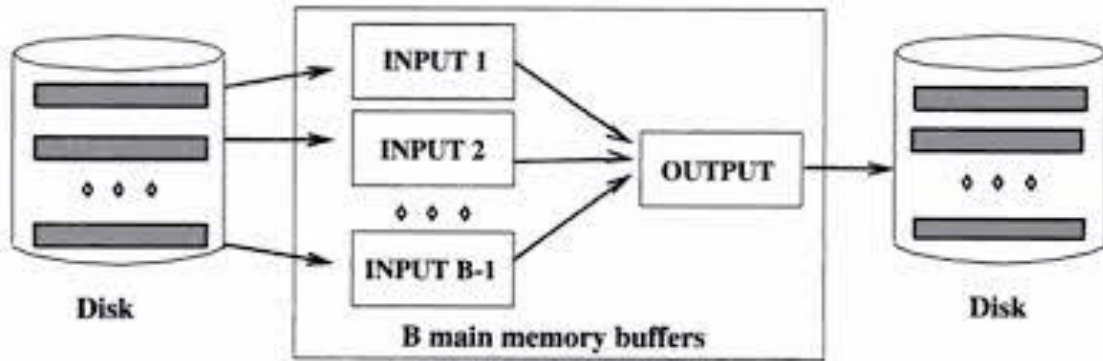


Blocked Sort-Based Indexing (BSBI)

- Pero es más eficiente hacer una fusión de múltiples mezclas, donde se está leyendo todos los bloques simultáneamente:
 - Abra todos los archivos de bloque simultáneamente y mantenga un búfer de lectura para cada uno y un búfer de escritura para el archivo de salida.
 - En cada iteración, seleccione el ID del término más bajo que no se haya procesado utilizando una cola de prioridad.
 - Combine todas las listas de publicaciones para ese termID y escríbalo al disco.

Blocked Sort-Based Indexing (BSBI)

- Pero es más eficiente hacer una fusión de múltiples mezclas, donde se está leyendo todos los bloques simultáneamente:



Blocked Sort-Based Indexing (BSBI)

- **Problemas pendientes con éste algoritmo:**

- Hemos trabajado bajo el supuesto de:
 - **Podemos mantener el diccionario en la memoria principal.**
- Entonces necesitamos que el diccionario (el cual crece dinámicamente) al ser implementado soporte eficientemente el mapeo de <term – termID>.
 - O mejor aun, trabajar directamente con el term.

Single-pass in-memory indexing (SPIMI)

- Generar diccionarios (hash) separados para cada bloque, sin necesidad de mantener el mapeo <term-termID> entre los bloques.
- No ordenar (slide 38). Acumular en el hash las publicaciones en listas a medida que van ocurriendo.
- De esta manera, podemos generar un índice invertido completo para cada bloque.
- Estos índices separados pueden luego ser mezclados en un big index.

Single-pass in-memory indexing (SPIMI)

BSBINDEXCONSTRUCTION()

```
1   $n \leftarrow 0$ 
2  while (all documents have not been processed)
3  do
4       $n \leftarrow n + 1$ 
5       $token\_stream \leftarrow ParseDocs( )$ 
6       $fn \leftarrow SPIMI-INVERT(token\_stream)$ 
7  MERGEBLOCKS( $f_1, \dots, f_n; f_{merged}$ )
```

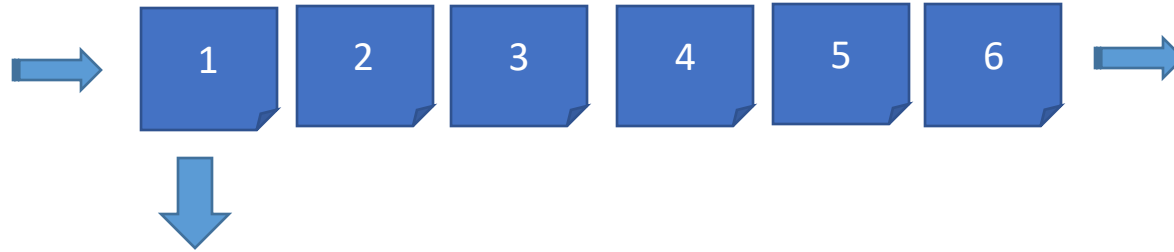

Single-pass in-memory indexing (SPIMI)

SPIMI-INVERT(*token_stream*)

```
1  output_file = NEWFILE()
2  dictionary = NEWHASH()
3  while (free memory available)
4  do token ← next(token_stream)
5      if term(token) ∉ dictionary
6          then postings_list = ADDTOdictionary(dictionary, term(token))
7          else postings_list = GETPOSTINGSList(dictionary, term(token))
8      if full(postings_list)
9          then postings_list = DOUBLEPOSTINGSList(dictionary, term(token))
10     ADDTOPOSTINGSList(postings_list, docID(token))
11 sorted_terms ← SORTTERMS(dictionary)
12 WRITEBLOCKTODISK(sorted_terms, dictionary, output_file)
13 return output_file
```

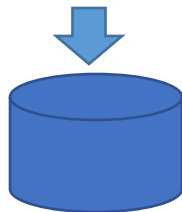
Luego, la mezcla de bloques es análogo a BSBI →

Single-pass in-memory indexing (SPIMI)



Local Dictionary

key	Posting List
w1	[doc1, doc3, doc5,...]
w2	[doc2, doc4, doc8, doc9,...]



Posting Lists: mantiene un tamaño inicial fijo, si este se llena, duplica su tamaño.

Si el Local Dictionary se llena, se vacía al disco.

Local Dictionary

key	df	Bucket
w1		B1
w2		B2

[doc1,
doc3,
doc5,...]

[...]

key	df	Bucket
w2		B3
w3		B4

[doc1,
doc3,
doc5,...]

[doc6,
doc9,
doc12]

[...]

key	df	Bucket
w1		B6
w4		B7

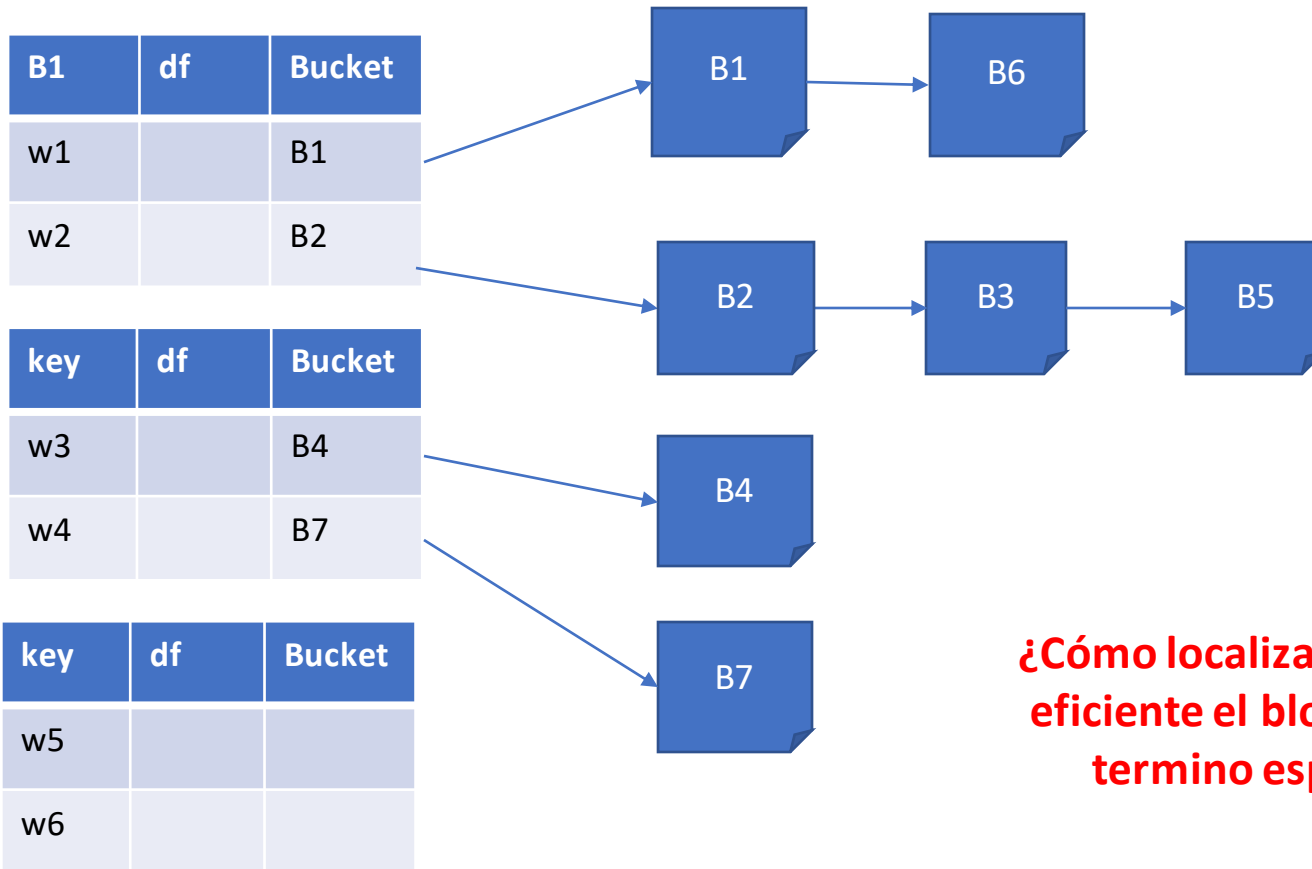
[...]

[...]

Los Posting List pueden almacenarse en Buckets independiente.

Ventaja: diccionarios locales pueden almacenar mas términos ordenados, acelerando el proceso del merge.

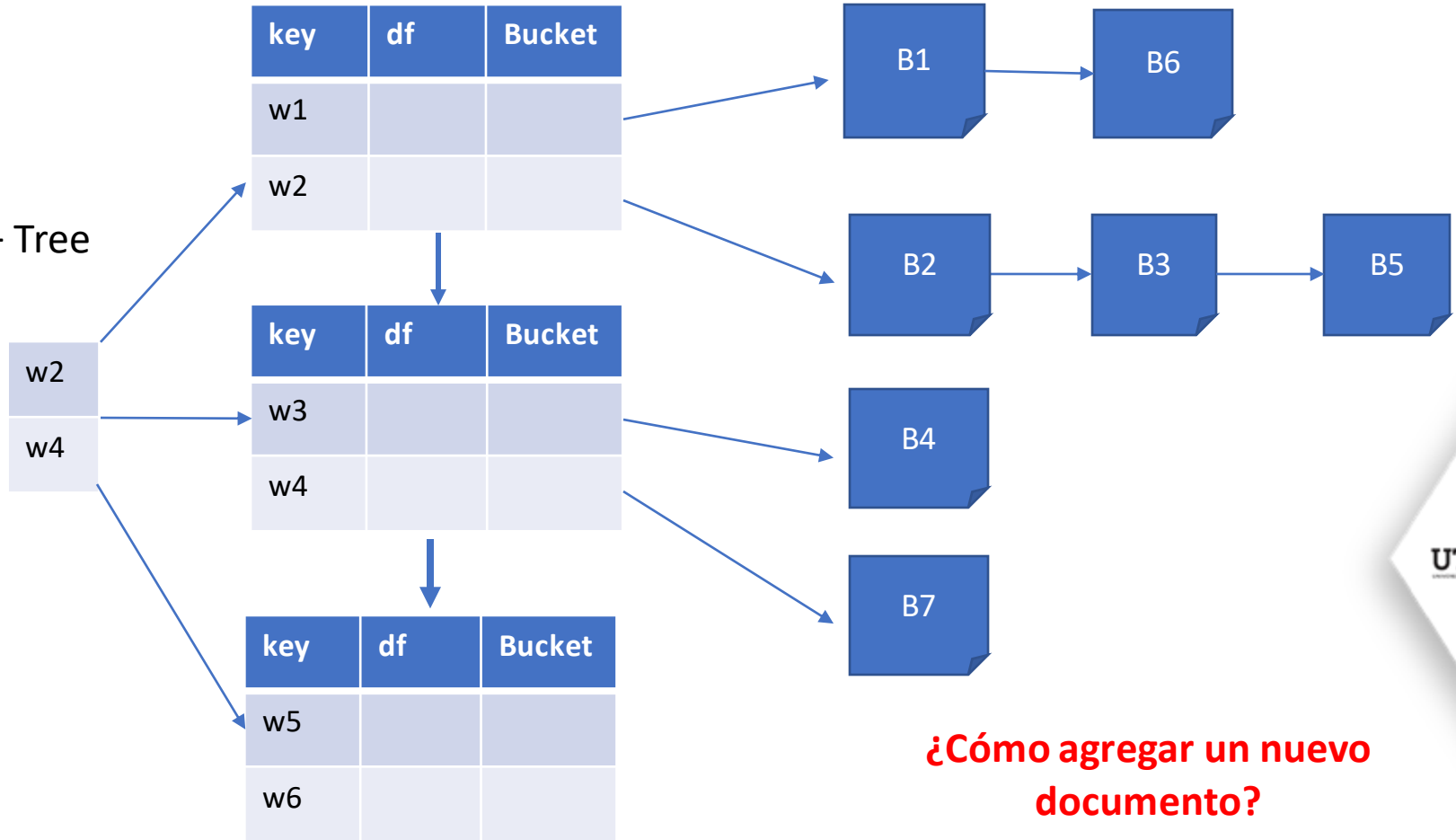
Merge Blocks



¿Cómo localizar de manera eficiente el bloque de un término específico?

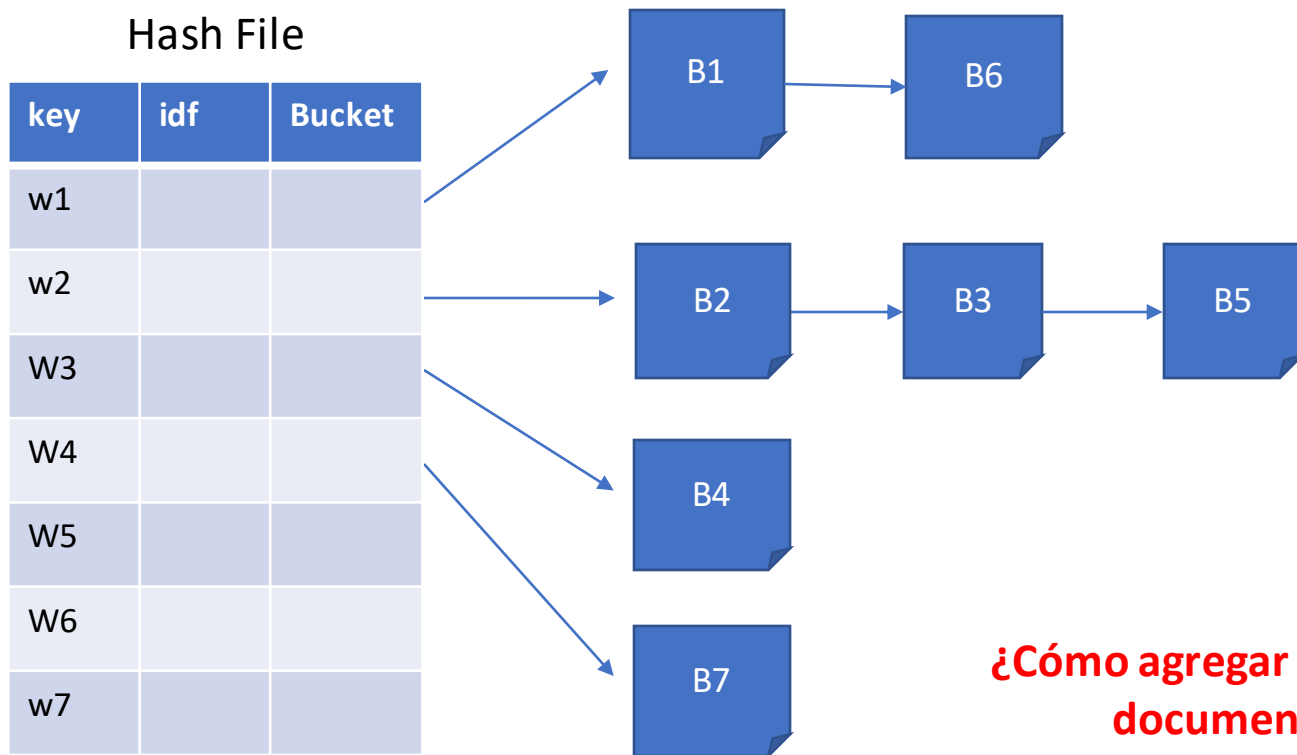
Alternativa de Implementación con B+ Tree

B+ Tree



¿Cómo agregar un nuevo documento?

¿Implementación con Extendible Hashing?



¿Cómo agregar un nuevo documento?

6.



Modelos de Recuperación de Texto

Implementaciones

Indexing for Full-text Search in PostgreSQL

GIN

```
CREATE INDEX content_idx ON News USING gin(content);
```

INDEX	
the	[1,3,4]
quick	[1]
brown	[1]
fox	[1]
jumps	[2]
over	[2]
lazy	[3,4]
dog	[3,4]
is	[5]
sleeping	[5]

TABLE	
1	the quick brown fox
2	jumps over
3	the lazy dog
4	because the lazy dog
5	is sleeping

Generalized Inverted Index

PostgreSQL crea el índice a partir de la representación vectorial generado de cada tupla en el campo textual especificado.

<https://postgrespro.com/blog/pgsql/4261647>

Indexing for Full-text Search in PostgreSQL

`select * from News where
to_tsquery('english', 'keyword1 & keyword2')
@@ content;`

Year	Similarity	Author	Article	# of words	Detail
2018	1.0	Casals	brain computer interface with corrupted eeg data a tensor completion approach eeg data tensor completion rehabilitation engineering missing channels performance	# of words: 29667	Abstract
2018	1.0	Castro	inferno inference aware neural optimisation neural network approximate bayesian computation expected variance class/utf01er nuisance parameter	# of words: 5400	Abstract
2018	1.0	Chen	on landscape of lagrangian function and stochastic search for constrained nonconvex optimization >symu2713iru2326 oea>oea delufb01nedcase	# of words: 31248	Abstract

INDEX	
the	[1,3,4]
quick	[1]
brown	[1]
fox	[1]
jumps	[2]
over	[2]
lazy	[3,4]
dog	[3,4]
is	[5]
sleeping	[5]



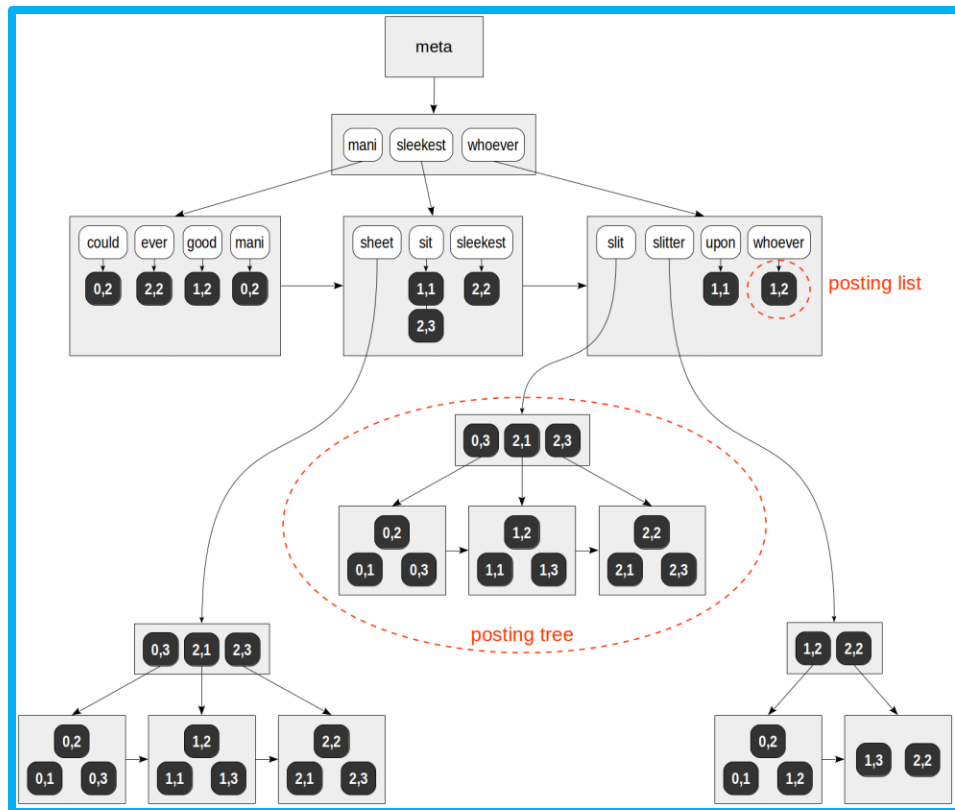
Indexing for Full-text Search in PostgreSQL

GiST:

- Índice dinámico (inserciones y eliminaciones)
- Adaptable para grandes volúmenes de datos.

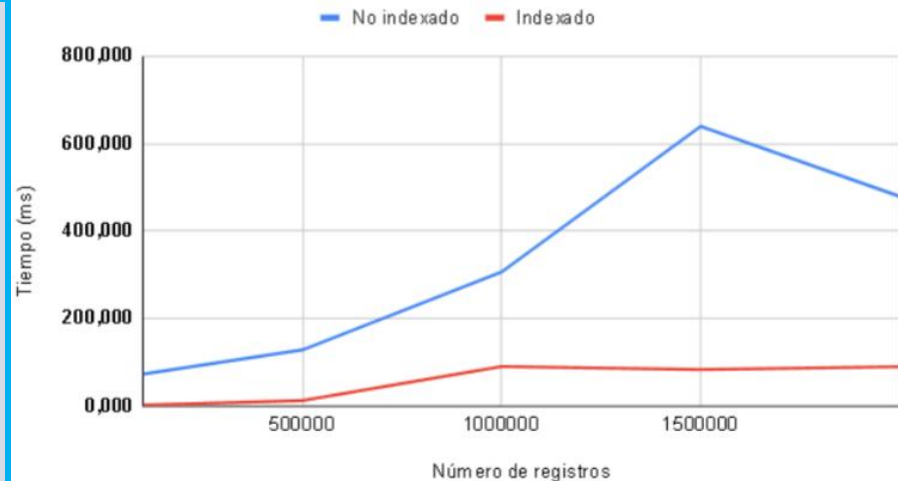
```
CREATE INDEX docs_gist_idx  
ON news  
USING GIST (to_tsvector('spanish', content));
```

```
SELECT * FROM news  
WHERE to_tsvector('spanish', content) @@  
plainto_tsquery('spanish', 'inteligencia  
artificial')  
LIMIT 10;
```



Desempeño del índice invertido usando Like

```
CREATE TABLE articles (  
  body text,  
  body_indexed text  
);  
  
-- create an index  
CREATE INDEX articles_search_idx ON articles USING gin  
(body_indexed gin_trgm_ops);  
  
-- populate table with data  
insert into articles  
select md5(random()::text)  
from (select * from generate_series (1,10000) as id) as T;  
update articles set body_indexed = body;  
  
-- non-indexed test  
explain analyze  
SELECT count(*) FROM articles where body ilike '%abcd%';  
-- indexed test  
explain analyze  
SELECT count(*) FROM articles where body_indexed ilike '%abcd%';
```



Desempeño del índice invertido en PostgreSQL

```
create table articles (  
    num integer, id integer,  
    title text, publication varchar(50),  
    author text, date varchar(12), year float,  
    month float, url text, content text  
);  
-- load data from a csv file  
COPY articles FROM '/home/datasets/articles1.csv'  
    WITH CSV HEADER;  
-- add indexed column  
ALTER TABLE articles ADD COLUMN full_text tsvector  
    GENERATED ALWAYS AS  
    (to_tsvector('english', title || ' ' || content)) STORED;  
-- create an index  
CREATE INDEX full_text_gin ON articles USING GIN (full_text_idx);  
  
-- non-indexed test  
EXPLAIN ANALYZE  
SELECT title, content FROM articles  
    WHERE full_text @@ to_tsquery('english', 'York | Trump');  
-- indexed test  
EXPLAIN ANALYZE  
SELECT title, content FROM articles  
    WHERE full_text_idx @@ to_tsquery('english', 'York | Trump');
```



GIN vs GiST

- GiST es mejor para aplicaciones con muchas escrituras y que requieren flexibilidad, como sistemas de gestión de contenido.
- GIN es más rápido en operaciones de búsqueda, lo que lo hace perfecto para motores de búsqueda y consultas sobre grandes volúmenes de datos, aunque su construcción y actualización son más costosas en términos de rendimiento.
- Ambos son índices eficientes pero su elección depende del equilibrio entre velocidad de búsqueda y frecuencia de actualización.

MongoDB para Full text Search

Dictionary

① PAELLA CHICKEN SEAFOOD

② BRAISE LAMB SHANK

③ CHICKEN KING

Index

BRAISE ②

CHICKEN ① ③

KING ③

LAMB ②

PAELLA ①

SEAFOOD ①

SHANK ②

- **MongoDB** ofrece soporte nativo para realizar búsquedas eficientes en datos textuales, permitiendo **consultas** escritas en **lenguaje natural**.
- El sistema gestiona automáticamente el procesamiento del texto y la generación de índices invertidos para optimizar el rendimiento de las búsquedas.

```
db.articulos.createIndex({ "campoTexto": "text" })
```

```
db.articulos.find({ $text:  
  { $search: "big data y análisis de datos",  
    $caseSensitive: true } })
```

Usa un versión mejorada de TF-IDF: BM25

Solr: Índice Textual + SQL



- Dashboard
- Logging
- Core Admin
- Java Properties
- Thread Dump
- CDataCore
 - Overview
 - Analysis
 - Dataimport
 - Documents
 - Files
 - Ping
 - Plugins / Stats
 - Query
 - Replication
 - Schema
 - Segments info

Request-Handler (qt)

/select

— common

q

,

fq

sort

start, rows

0

10

fl

df

Raw Query Parameters

key1=val1&key2=val2

wt

☐ indent off

☐ debugQuery

☐ dismax

☐ edismax

☐ hl

☐ facet

☐ spatial

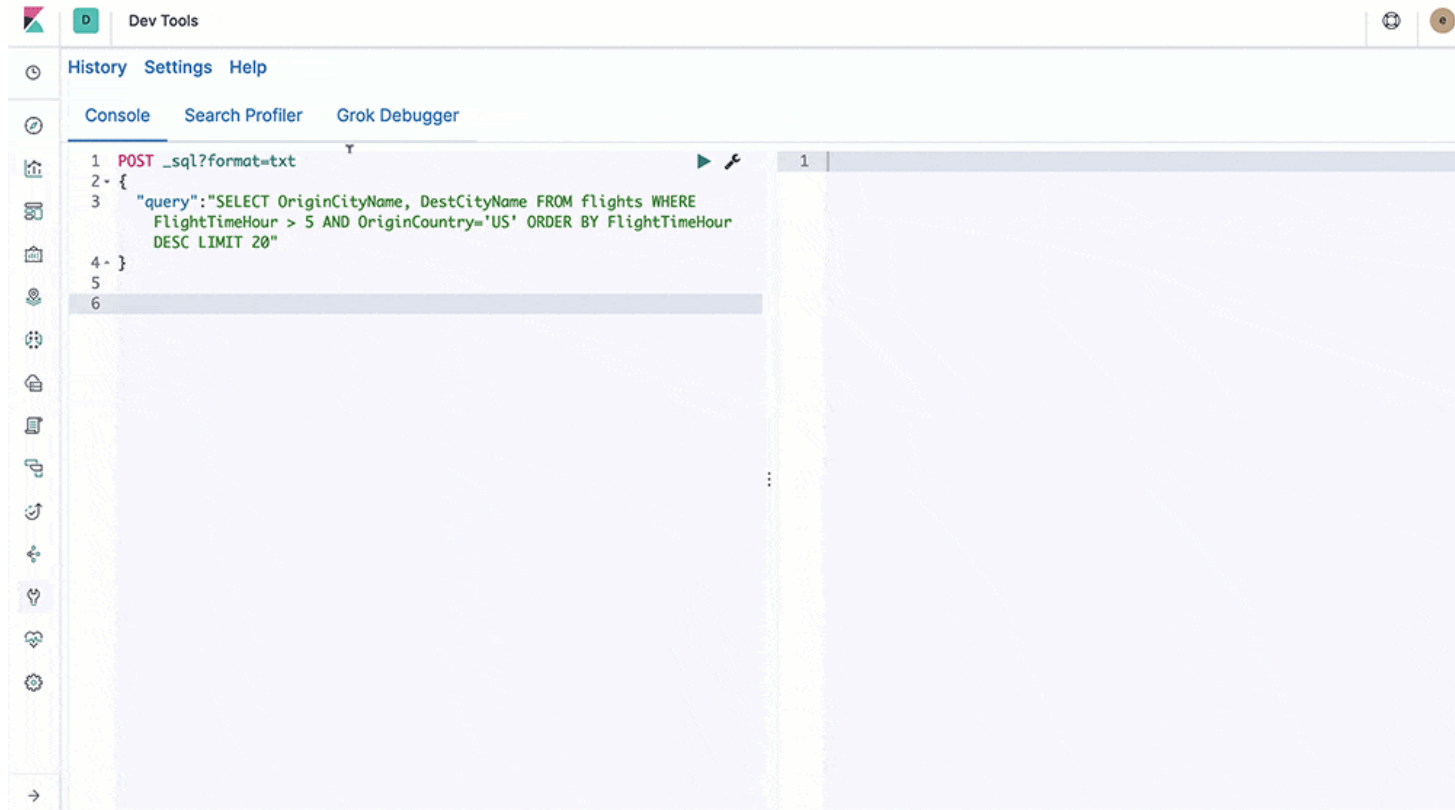
☐ spellcheck

Execute Query

http://localhost:8983/solr/CDataCore/select?q=**%3A*

```
{
  "responseHeader": {
    "status": 0,
    "QTime": 0,
    "params": {
      "q": "**%3A*",
      "_:": "1593752320952"
    }
  },
  "response": {
    "numFound": 12, "start": 0, "docs": [
      {
        "LastModifiedDate": "2020-07-01T02:15:38Z",
        "Description": "Genomics company engaged in mapping and sequencing of the human genome and developing gene-based c",
        "Phone": "(650) 867-3450",
        "CreatedDate": "2020-07-01T02:15:38Z",
        "Website": "www.genepoint.com",
        "id": "0012x000009VGSvAA0",
        "Fax": "(650) 867-8885",
        "Name": "GenePoint",
        "_version_": 1671171128570150912,
      },
      {
        "LastModifiedDate": "2020-07-01T02:15:38Z",
        "Phone": "+44 191 4956209",
        "CreatedDate": "2020-07-01T02:15:38Z",
        "Website": "http://www.uos.com",
        "id": "0012x000009VGSvAA0",
        "Fax": "+44 191 495620",
        "Name": "United Oil & Gas, UK",
        "_version_": 167117112859316736,
      },
      {
        "LastModifiedDate": "2020-07-01T02:15:38Z",
        "Phone": "(650) 450-8810",
        "CreatedDate": "2020-07-01T02:15:38Z",
        "Website": "http://www.uos.com",
        "id": "0012x000009VGSvAA0",
        "Fax": "(650) 450-8820",
        "Name": "United Oil & Gas, Singapore",
        "_version_": 1671171128597413888,
      },
      {
        "LastModifiedDate": "2020-07-01T02:15:38Z",
        "Description": "Edge, founded in 1998, is a start-up based in Austin, TX. The company designs and manufactures a c",
        "Phone": "(512) 757-6000",
      }
    ]
  }
}
```

Elastic Search: Índice Textual + SQL



The screenshot shows the DevTools console in a web browser. The 'Console' tab is active, displaying a POST request to the endpoint `_sql?format=txt`. The request body is a JSON object containing a SQL query. The query is: `"query": "SELECT OriginCityName, DestCityName FROM flights WHERE FlightTimeHour > 5 AND OriginCountry='US' ORDER BY FlightTimeHour DESC LIMIT 20"`. The console interface includes a left sidebar with various DevTools icons, a top bar with 'History', 'Settings', and 'Help' tabs, and a right sidebar with 'Search Profiler' and 'Grok Debugger' tabs. The main area shows the request details, including the method, URL, and body.

```
1 POST _sql?format=txt
2 {
3   "query": "SELECT OriginCityName, DestCityName FROM flights WHERE
4     FlightTimeHour > 5 AND OriginCountry='US' ORDER BY FlightTimeHour
5     DESC LIMIT 20"
6 }
```


Laboratorio 9

